Activity 1

```
# @title Activity 1
graph = {
    '5' : ['3','7'],
    '3' : ['2', '4'],
    '7' : ['8'],
    '2' : [],
    '4' : ['8'],
    '8' : []
}

visited = []
queue = []

def bfs(visited, graph, node):
    visited.append(node)
    queue.append(node)

    while queue:
        m = queue.pop(0)
        print(m, end=" ")

        for neighbour in graph[m]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

print("Following is the Breadth-First Search")
bfs(visited, graph, '5')
```

```
Following is the Breadth-First Search
5 3 7 2 4 8
```

Activity 2

Show code

```
Parents:
1 -> Root
2 -> 1
5 -> 2
6 -> 2
3 -> 1
4 -> 1
7 -> 4

Children:
1 ->  2 3 4
2 ->  5 6
3 ->
4 ->  7
5 ->
6 ->
7 ->

Leaf Nodes:
3 5 6 7

Degrees:
1 : 3
2 : 2
3 : 0
4 : 1
5 : 0
6 : 0
7 : 0
```

Activity 3

```
# @title Activity 3
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
```

```
    'C': ['F'],
    'D': [],
    'E': ['G'],
    'F': [],
    'G': []
}

visited = []
queue = []

bfs(visited, graph, 'A')
```

```
A B C D E F G
```

Activity 4

```
# @title Activity 4
# Graph from Activity 1 / 3
graph = {
    '5' : ['3','7'],
    '3' : ['2', '4'],
    '7' : ['8'],
    '2' : [],
    '4' : ['8'],
    '8' : []
}

# BFS Function
def bfs(graph, start):
    visited = []
    queue = []

    visited.append(start)
    queue.append(start)

    while queue:
        node = queue.pop(0)
        print(node, end=" ")

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# --------- Activity 4 Runs ---------

print("Start Node = 5")
bfs(graph, '5')

print("\n\nStart Node = 3")
bfs(graph, '3')

print("\n\nStart Node = 7")
bfs(graph, '7')

print("\n\nStart Node = 2")
bfs(graph, '2')
```

```
Start Node = 5
5 3 7 2 4 8

Start Node = 3
3 2 4 8

Start Node = 7
7 8

Start Node = 2
2
```